# Diagramme cas d’utilisation :

## 1. Objectif du Diagramme

- Le diagramme de cas d'utilisation sert à **structurer et modéliser les besoins (les exigences)** des utilisateurs vis-à-vis d'un système.
- Il définit le comportement du système **du point de vue d'un utilisateur extérieur**, sans se soucier de la technique ou de la façon dont le code sera écrit (approche "boîte noire").

## 2. Les Concepts Clés

- **L'Acteur :** Représente un rôle joué par une personne, un autre système ou un processus externe qui interagit directement avec l'application. On le dessine avec le petit bonhomme (stickman).
- **Le Cas d'Utilisation (Use Case) :** C'est une unité cohérente de fonctionnalités qui rend un service complet et visible à l'acteur. On le représente par une ellipse (un ovale) contenant un verbe à l'infinitif.
- **Le Système (Le cadre) :** Une boîte rectangulaire qui délimite la frontière entre ce qui est à l'intérieur du logiciel (les cas d'utilisation) et ce qui est à l'extérieur (les acteurs).

## 3. Les Relations entre Cas d'Utilisation

C'est le cœur théorique de la page (ce qu'on a appliqué sur ton schéma) :

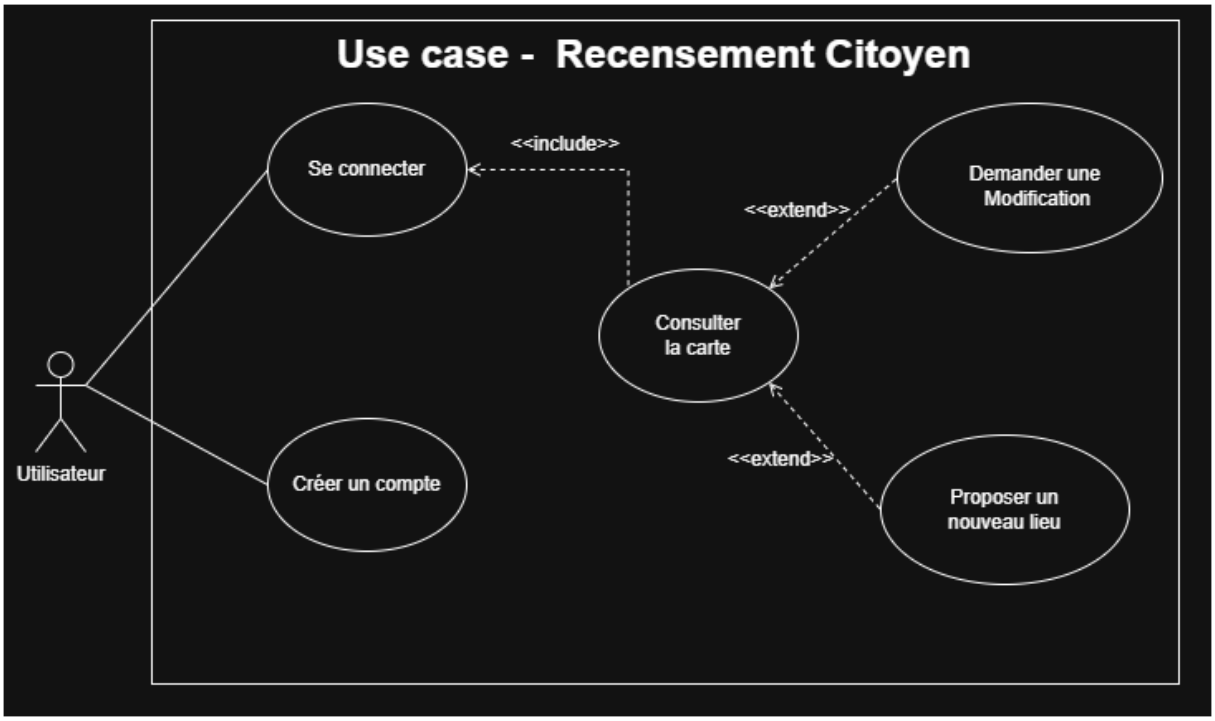
- **L'Inclusion (`«include»`) :** Le cas A inclut le cas B. Cela signifie que le comportement de B est **obligatoire** pour réaliser A. *Exemple du cours : Retirer de l'argent* `«include»` *S'authentifier.* La flèche va de A vers B.
- **L'Extension (`«extend»`) :** Le cas B étend le cas A. Cela signifie que le comportement de B est **optionnel** ou soumis à une condition particulière. Il vient enrichir le cas principal A. La flèche va de l'option B vers le cas principal A (sens inverse de l'inclusion).
- **La Généralisation/Héritage :** Un cas d'utilisation (ou un acteur) plus spécifique peut hériter d'un cas plus général (représenté par une flèche à bout triangulaire vide).

## 4. La Description Textuelle des Cas d'Usage

Le cours insiste sur le fait qu'un diagramme tout seul ne suffit pas. Chaque bulle doit être accompagnée d'une **description textuelle** (ce qu'on a fait à l'étape précédente) divisée en trois blocs :

1. **L'identification :** Nom, objectif, acteur principal, version.
2. **Le scénario nominal :** La suite d'étapes standard (le "chemin idéal") où tout se passe bien.
3. **Les scénarios alternatifs / d'exceptions :** Les variantes ou la gestion des erreurs (ex: mauvais mot de passe).

# Scénarios des Cas d'Usage




## 1. Créer un compte

- **Acteur principal :** Utilisateur
- **Préconditions :** L'utilisateur n'est pas inscrit.
- **Scénario nominal :**
    1. L'utilisateur clique sur "Créer un compte".
    2. Le système affiche le formulaire d'inscription.
    3. L'utilisateur saisit son nom, son adresse e-mail et son mot de passe, puis valide.
    4. Le système vérifie la validité des informations (et l'unicité de l'email).
    5. Le système enregistre le nouvel utilisateur et affiche un message de succès.
- **Scénario alternatif (Erreur) :** Si l'email est déjà utilisé à l'étape 4, le système affiche un message d'erreur et invite à se connecter ou changer d'email.

## 2. Se connecter

- **Acteur principal :** Utilisateur
- **Préconditions :** L'utilisateur possède déjà un compte.
- **Scénario nominal :**
    1. L'utilisateur clique sur "Se connecter".
    2. Le système affiche le formulaire de connexion.
    3. L'utilisateur saisit son e-mail et son mot de passe, puis valide.
    4. Le système vérifie les identifiants en base de données.
    5. Les identifiants sont corrects, le système connecte l'utilisateur et le redirige vers la carte.

## 3. Consulter la carte

- **Acteur principal :** Utilisateur
- **Préconditions :** L'utilisateur doit obligatoirement être authentifié.
- **Scénario nominal :**
    1. L'utilisateur accède à l'application.
    2. Le système vérifie que la session est active (ou force la connexion).
    3. Le système charge les coordonnées des lieux ainsi que le tracé des zones géographiques.
    4. Le système affiche la carte interactive avec les filtres par catégories (fontaines, parkings vélos...).

## 4. Proposer un nouveau lieu / Demander une modification

- **Acteur principal :** Utilisateur
- **Préconditions :** L'utilisateur est en train de consulter la carte.
- **Scénario nominal :**
    1. L'utilisateur clique sur un bouton "Ajouter un lieu" (ou clique sur un lieu existant pour le modifier).
    2. Le système ouvre un formulaire contextuel (coordonnées GPS pré-remplies, choix du type de lieu ou saisie des corrections).
    3. L'utilisateur valide sa proposition.
    4. Le système enregistre l'action dans la table des demandes avec le statut "En attente de validation".
    5. Le système confirme la prise en compte à l'utilisateur.

# Dictionnaire de Données — Projet NumCity

| Code mnémonique | Nom | Type SQL | Contraintes | Description |
|---|---|---|---|---|
| User_id | Identifiant utilisateur | SERIAL | PRIMARY KEY | Clé primaire auto-incrémentée unique pour chaque citoyen. |
| User_name | Nom d'utilisateur | VARCHAR(20) | NOT NULL | Nom ou pseudo de l'utilisateur inscrit. |
| User_email | Email d'utilisateur | VARCHAR(100) | NOT NULL, UNIQUE | Email servant d'identifiant de connexion (doublons interdits). |
| User_mdp | Mot de passe d'utilisateur | VARCHAR(100) | NOT NULL | Empreinte sécurisée (hachée) du mot de passe. |
| Lieu_id | Identifiant lieu | SERIAL | PRIMARY KEY | Clé primaire unique pour chaque lieu recensé. |
| Lieu_latitude | Coordonnées d'un lieu | NUMERIC(9 ,6) | NOT NULL | Coordonnée GPS de latitude |
| Lieu_longtitude | Coordonnées d'un lieu | NUMERIC(9 ,6) | NOT NULL | Coordonnée GPS de longitude |
| Demande_id | Identifiant d'une demande | SERIAL | PRIMARY KEY | Clé primaire unique de suivi des requêtes citoyennes. |
| Demande_type | Type de demande | VARCHAR(20) | NOT NULL | Nature de l'action : 'Proposition', 'Modification'. |
| Demande_description | Description de la demande | TEXT | | Description textuelle des changements demandés (optionnel). |
| Demande_date | Date de la demande | TIMESTAMP | NOT NULL | Date et heure de soumission (par défaut CURRENT_TIMESTAMP). |
| Catégorie_id | Identifiant catégorie de lieu | SERIAL | PRIMARY KEY | Clé primaire unique de la catégorie de lieu. |
| Catégorie_nom | Nom de catégorie | VARCHAR(50) | NOT NULL | Le type d'un lieu (points de parking pour vélos et trottinettes, points de recharge électrique, fontaines à eau…) |
| Zone_id | Identifiant de la zone géographique | SERIAL | PRIMARY KEY | Clé primaire unique de la zone. |
| Zone_nom | Nom de la zone géographique | VARCHAR(50) | NOT NULL | Non de la rue ou de la zone |

# 1. Qu'est-ce que la méthode Merise ?

- C'est une méthode de conception et de réalisation de projets informatiques, spécifiquement adaptée aux **bases de données relationnelles**.
- Sa grande force est la **séparation des données et des traitements**. On modélise ce que la base doit contenir (les données) indépendamment de la façon dont l'application va tourner (les traitements).
- Le cours se concentre sur le cycle de conception en deux grandes étapes : le **MCD** (abstrait, humain) puis le **MLD** (technique, logique).

# 2. Le Modèle Conceptuel de Données (MCD)

Le MCD est la représentation graphique de l'esprit du système. Il utilise trois concepts fondamentaux :

- **L'Entité (ou Individu) :** Représente un objet concret ou abstrait du monde réel (ex: *Utilisateur*, *Lieu*). Elle possède des **propriétés (attributs)**.
- **L'Identifiant :** Chaque entité doit obligatoirement posséder une propriété unique (soulignée) qui permet de distinguer chaque ligne à coup sûr (ex: *uti_id*).
- **L'Association (ou Relation) :** C'est le lien verbal entre deux ou plusieurs entités (représenté par un ovale, ex: *[Appartenir]*, *[Soumettre]*).

### Les Cardinalités

Elles définissent le nombre de fois minimum et maximum qu'une entité peut participer à une association.

- **0,1 :** Au minimum aucune fois, au maximum une seule fois.
- **1,1 :** Au minimum une fois, au maximum une seule fois (obligation stricte).
- **0,N :** Au minimum aucune fois, au maximum plusieurs fois.
- **1,N :** Au minimum une fois, au maximum plusieurs fois.

# 3. Le Modèle Logique de Données (MLD)

Le MLD est la traduction technique du MCD sous forme de tables prêtes pour le SQL. Le cours explique les **règles de transformation universelles** :

### Règle 1 : Les Entités deviennent des Tables

- Chaque entité du MCD devient une table.
- L'identifiant de l'entité devient la **Clé Primaire (Primary Key - PK)** de la table.

### Règle 2 : La relation de type 1:N (ou 0:N) — La migration de clé

- Quand on a une relation avec une cardinalité maximale à **1** d'un côté et à **N** de l'autre (ex: *Lieu (1,1) --- Appartenir --- (0,N) Catégorie*).
- **La règle :** La clé primaire du côté **N** (*cat_id*) migre comme **Clé Étrangère (Foreign Key - FK)** du côté **1** (dans la table *Lieu*).

### Règle 3 : La relation de type N:N (Plusieurs à Plusieurs) — La table d'association

- Quand on a une cardinalité maximale à **N** des deux côtés (ex: *Utilisateur (0,N) --- Favoris --- (0,N) Lieu*).
- **La règle :** L'association disparait et se transforme en une **nouvelle table de liaison** (ex: table *favoris*). Cette table intermédiaire contient les clés primaires des deux entités concernées (*uti_id* et *lie_id*) qui forment ensemble une clé primaire composée.

# Le modèle conceptuel de données MCD du projet NumCity:

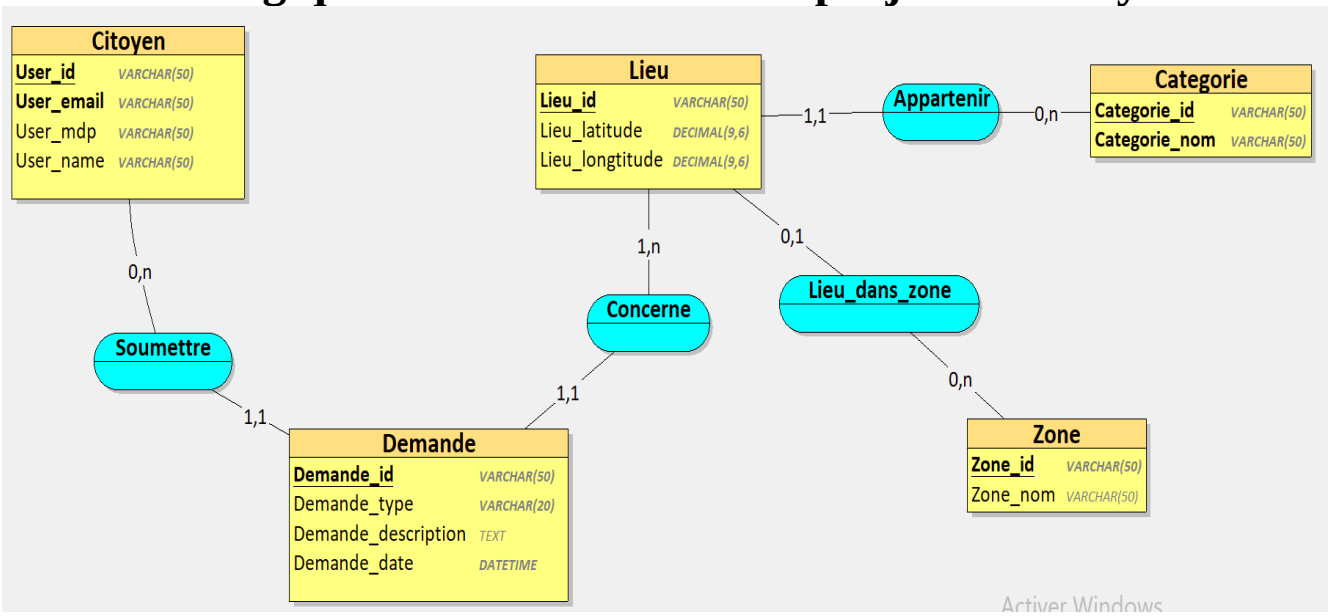
Le MCD formalise l'ensemble des règles métiers et des besoins conceptuels du projet NumCity sans contrainte technique. Il s'articule autour de **5 entités** interconnectées par des associations sémantiques précises :

- **Citoyen :** Représente les utilisateurs de l'application. Un citoyen possède un identifiant unique, un nom, un email (unique) et un mot de passe haché.
- **Demande :** Centralise toutes les requêtes citoyennes (propositions d'ajouts ou modifications).
- **Lieu :** Recense tous les points d'intérêt et équipements urbains de la ville via leurs coordonnées géographiques de latitude et de longitude.
- **Categorie :** Permet de classifier les types d'équipements disponibles (Fontaines, Toilettes, Parkings...).
- **Zone :** Représente les différents secteurs géographiques ou quartiers de la commune.

### Justification des choix de Cardinalités :

- **Relation [Soumettre] (`0,n - 1,1`) :** Un citoyen peut être inactif ou très engagé et soumettre de 0 à n demandes au cours du temps. En revanche, une demande est strictement nominative et doit être initiée par `1,1` citoyen unique (traçabilité complète).
- **Relation [Concerne] (`1,1 - 1,n`) :** Une demande vise obligatoirement `1,1` lieu unique pour être traitée. Par contre, un même lieu physique peut faire l'objet de plusieurs demandes d'améliorations ou de signalements successifs au cours de sa vie (`1,n`).
- **Relation [Appartenir] (`1,1 - 0,n`) :** Pour la clarté de la carte, un lieu est rattaché à `1,1` seule catégorie. À l'inverse, une catégorie peut regrouper de 0 à n points d'intérêt sur le territoire.
- **Relation [Lieu_dans_zone] (`0,1 - 0,n`) :** Un quartier englobe naturellement plusieurs infrastructures (`0,n`). Toutefois, un équipement peut être positionné en périphérie ou en dehors de tout secteur administrativement cartographié par la mairie, d'où la cardinalité minimale à 0 (`0,1`).

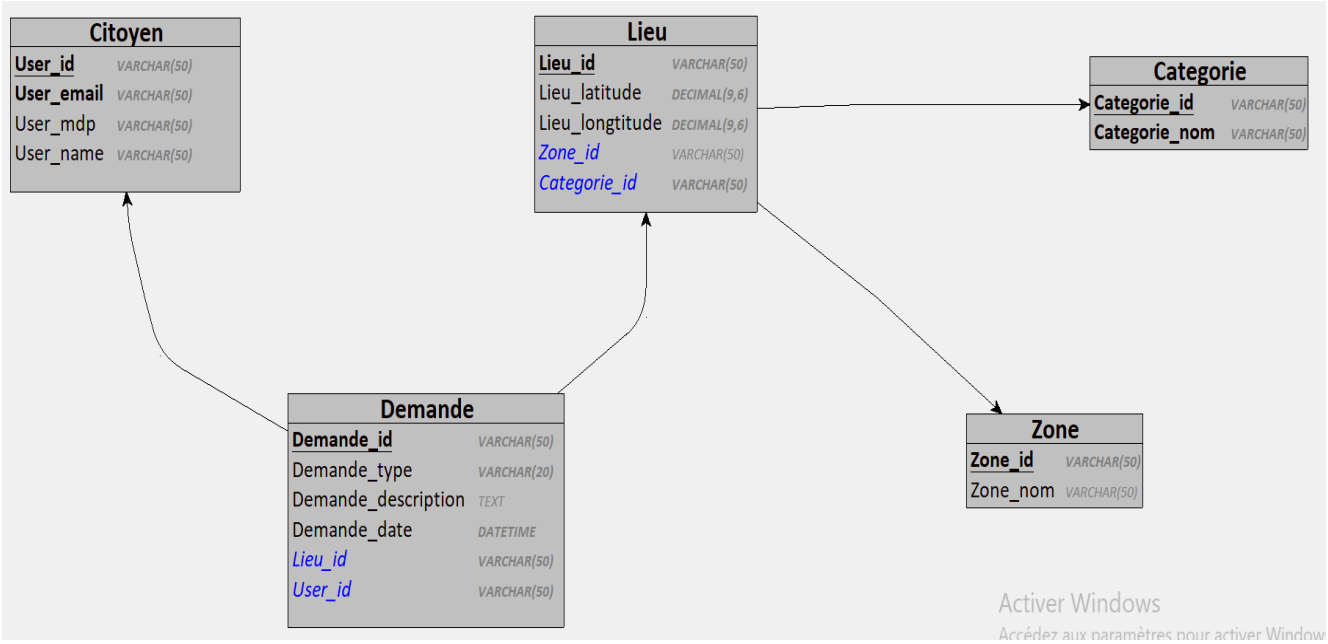
# Le modèle logique de données MLD du projet NumCity:




Le passage au MLD (représenté sur la figure `Capture.PNG`) traduit le modèle conceptuel en structures de tables prêtes pour l'implémentation relationnelle. Looping a appliqué les règles de dérivation Merise de manière optimale :

- **Transformation des Entités en Tables :** Les 5 entités du MCD deviennent les 5 tables de notre base de données. Les identifiants soulignés se transforment en **Clés Primaires**.
- **Migration des relations de type 1:N (ou 0:N) :** * Dans le MCD, les entités `Demande` et `Lieu` se trouvaient du côté des cardinalités maximales à **1** (via les liens *Soumettre*, *Concerne*, *Appartenir* et *Lieu_dans_zone*).
    - **Conséquence directe :** Ces tables ont hérité des identifiants des entités parentes sous forme de **Clés Étrangères** (représentées en italique bleu sur le schéma).
    - La table `Demande` intègre ainsi `User_id` et `Lieu_id` pour lier directement l'action à l'acteur et à l'objet.
    - La table `Lieu` intègre `Categorie_id` et `Zone_id` pour structurer l'arborescence et l'emplacement des données.

Cette structure optimisée garantit l'absence de tables de liaison superflues, ce qui allège grandement le code SQL tout en assurant une intégrité référentielle absolue.




# Normalisation et Formes Normales :

Pour garantir la cohérence des données, éviter les redondances et optimiser les performances globales de la base de données de NumCity, le modèle logique de données a été soumis aux trois premières formes normales (FN).

## 1. Première Forme Normale (1FN)

- **Règle théorique :** Une table est en 1FN si tous ses attributs sont **atomiques** (indivisibles) et s'il existe au moins une clé primaire unique. Chaque intersection ligne/colonne ne doit contenir qu'une seule valeur.
- **Application au projet : Validé.** Les coordonnées géographiques des équipements ne sont pas stockées dans une seule chaîne de texte (ex: `"44.55,-4.75"`), mais ont été divisées en deux attributs atomiques distincts : `Lieu_latitude` et `Lieu_longtitude`. De plus, chaque table possède une clé primaire (`User_id`, `Lieu_id`, etc.) garantissant l'unicité des lignes.

## 2. Deuxième Forme Normale (2FN)

- **Règle théorique :** Une table est en 2FN si elle est en 1FN et si tous les attributs non-clés dépendent de la **totalité** de la clé primaire. Cela concerne surtout les tables ayant des clés primaires composées (plusieurs colonnes) : un attribut ne peut pas dépendre d'une seule partie de la clé.
- **Application au projet : Validé.** Nos tables principales (`Citoyen`, `Lieu`, `Demande`, `Zone`, `Categorie`) possèdent toutes des clés primaires simples (composées d'un seul attribut). Il n'y a donc aucun risque de dépendance partielle. Quant aux anciennes relations n:n, elles ont été simplifiées dans notre dernier modèle au profit de clés étrangères directes (comme `User_id` et `Lieu_id` dans `Demande`), éliminant tout conflit.

## 3. Troisième Forme Normale (3FN)

- **Règle théorique :** Une table est en 3FN si elle est en 2FN et si aucun attribut non-clé ne dépend d'un autre attribut non-clé. En clair : tous les attributs doivent dépendre **uniquement et directement** de la clé primaire (pas de dépendance transitive).
- **Application au projet : Validé.** Prenons l'exemple de la table `Citoyen` : l'attribut `User_name` dépend directement de `User_id`. Il ne dépend pas de `User_email` ni de `User_mdp`. Idem pour la table `Lieu` : la position GPS dépend uniquement de l'identifiant du lieu.